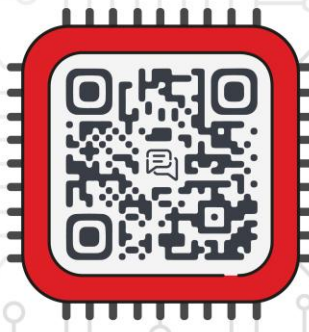




6



14



1680 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Binary heaps

د. عملي مشترك



RB Informatics; 12/5/2025

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

في محاضراتنا السابقة تناولنا مقدمةً في الأشجار بصورة عامة ثم تحدثنا عن الأشجار الثنائية بعد ذلك ناقشنا أحد أهم أشكال الأشجار الثنائية وهو شجرة البحث الثنائية ثم تحدثنا عن مفهوم التوازن ضمنها وكيف يمكن معالجته من خلال الـ (AVL) هنالك طرق أخرى ربما نتطرق لها بمحاضرات أخرى لاحقاً، واليوم سنقوم بالانتقال إلى نقطة جديدة ونوع جديد من الأشجار الثنائية وهو الـ BINARY HEAP

ما هو الـ BINARY HEAP:

هو شجرة ثنائية مكتملة (COMPLETED)، تحقق خاصية الـ HEAP وهذه الخاصية تختلف باختلاف نوع الـ heap، لدينا نوعان للـ HEAP: MIN HEAP and MAX HEAP

- في الـ MAX HEAP الخاصية هي: لكل عقدة ضمن الشجرة الثنائية يجب أن يتحقق أن تكون قيمة هذه العقدة أكبر أو تساوي قيمة كل من أبنائها.
- في الـ MIN HEAP الخاصية هي: لكل عقدة ضمن الشجرة الثنائية يجب أن يتحقق أن تكون قيمة هذه العقدة أصغر أو تساوي قيمة كل من أبنائها.

وهذا يجب أن يكون محقق عند كل شجرة جزئية من الشجرة الكبرى في كلا النوعين

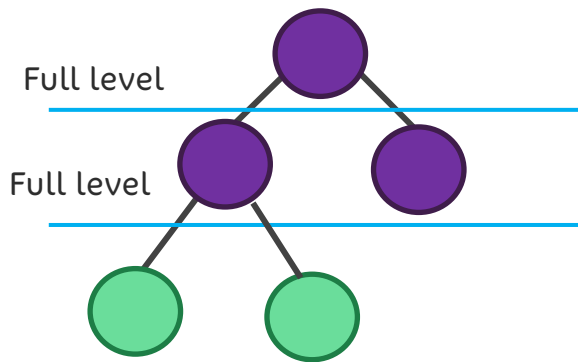
ملاحظة هامة:

العقد التي ليس لها أبناء تعتبر الـ HEAPS (الأوراق في الشجرة لا تخرق خاصية الـ heap التي ذكرناها أعلاه).

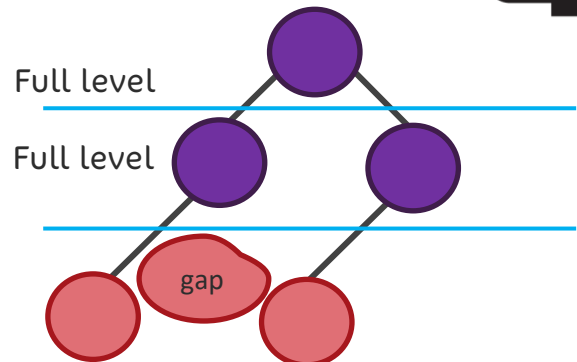
تحقيق ذلك يضمن أمراً مهماً: جذر الشجرة يحوي أكبر عنصر ضمن الشجرة في حالة الـ MAX HEAP، أو أصغر عنصر ضمن الشجرة في حالة الـ MIN HEAP سنستفيد من هذا الأمر بوقت لاحق.

تذكرة:

الـ COMPLETED BINARY TREE هي شجرة ثنائية جميع مستوياتها ممتلئة بالعقد عدا المستوى الأخير يكون ممتلئاً بدءاً من اليسار حتى نقطة معينة (يمكن أن يكون المستوى الأخير ممتلئاً كذلك لا ضرر) تصبح الـ COMPLETED TREE مثالية (PERFECT) (").



Valid level its filled starting from the left until a certain point



Invalid level its not filled starting from the left until a certain point (there is a gap)

في مقررنا عند استخدام مصطلح HEAP نحن نقصد تحديداً BINARY HEAP

٤ من المتعارف أن يتم استخدام المصفوفات في تمثيل الـ heaps، لما ذلك؟

كون الـ heap هو complete binary tree لن يكون هنالك هدر بالذاكرة في حال استخدمنا مصفوفة ستكون المصفوفة ممتلئة لحد ما وهذا سيكون أوفر من ناحية الذاكرة مقارنة باختزان المؤشرات ضمن عقد (كل مؤشر يستهلك 8 بايتات عادةً).

سنعتمد على تمثيل الشجرة باستخدام مصفوفة طبعاً سنقوم باستخدام "dynamic arrays /vector /arraylist /etc"

نظراً لكوننا نريد أن نتعامل بسهولة من ناحية الإضافة والحذف عندما نقوم بعمل البنية بالشكل العام (المصفوفات الديناميكية تعمل داخلياً من خلال استخدام مصفوفات ثابتة الحجم وعند تجاوز عدد العناصر لحد معين تقوم المصفوفة الديناميكية بعمل مصفوفة ثابتة ذات حجم جديد مناسب ونقل العناصر إليها) (راجع مقرر خوارزميات 1 لتفاصيل أكثر).

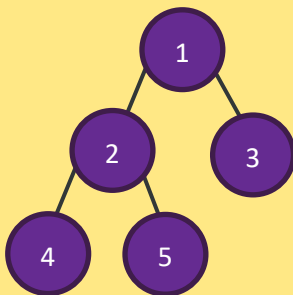
تذكرة:

لتمثيل شجرة ثنائية بمصفوفة أحادية، لكل عنصر في الـ index i ضمن المصفوفة:

- الابن اليسار يقع في $2*i+1$

- الابن اليمين يقع في $2*i+2$

و هذا بافتراض أن جذر الشجرة في الـ index 0



INDEX	0	1	2	3	4
VALUE	1	2	3	4	5



بناء الـ HEAP من مصفوفة معطاة

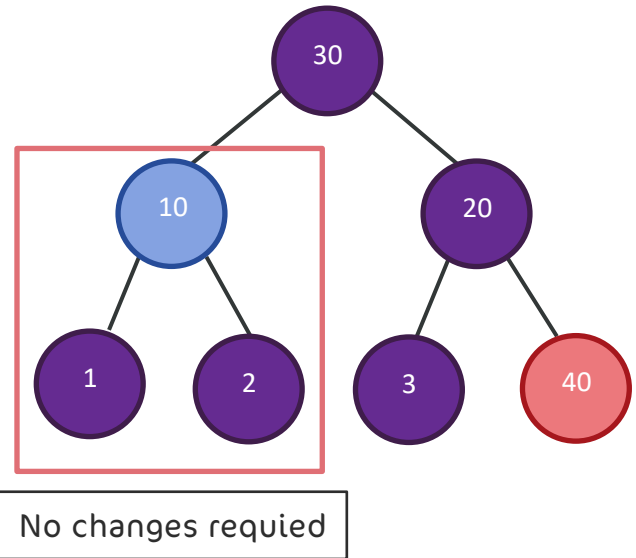
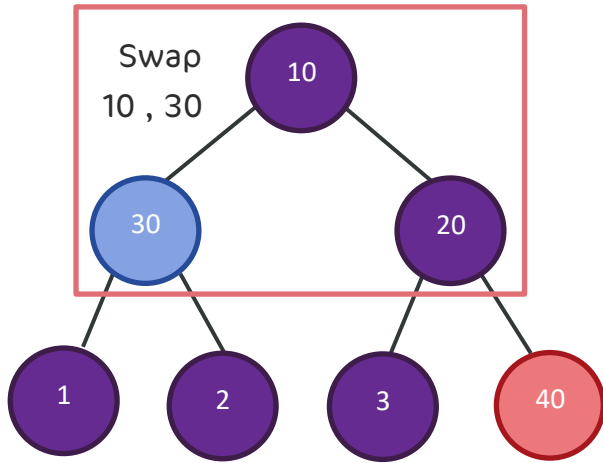
ليكن لدينا مصفوفة أحادية تمثل شجرة ثنائية completed قم بتحويلها لـ heap من خلال جعلها تحقق خاصية أحد نوعيه، سنختار الـ max heap مبدئياً (الـ min heap يقوم على نفس المبدأ مع عكس بعض الشروط).

الآن حتى نقوم بذلك لا بد لنا من فهم العملية الأساسية التي سنعتمد عليها وتسمى heapify، هذه العملية تهدف إلى إيصال العنصر الأكبر / الأصغر (حسب نوع الـ Heap) إلى جذر الشجرة الحالية، كيف نقوم بتنفيذها (بافتراض max heap)؟

- نقارن الجذر مع ابنه وفي حال كان أحد ابنيه أكبر من الجذر نبدل بين الجذر وبين الابن الأكبر (نريد إيصال العنصر الأكبر دوماً إلى جذر الشجرة التي نعمل عليها)، ثم نقوم بتكرار ذلك على الشجرة الجزئية المتأثرة.
 - في حال حصل تبديل مع أحد الأبناء؛ هذا يضمن الحفاظ على خاصية heap لجميع الأشجار الجزئية المتأثرة.
- في حال كنا نتحدث عن heap بصورة عامة ولدينا خلل في عقدة معينة فقط وكانت الأشجار الجزئية التابعة للعقدة التي فيها الخلل تحقق خواص الـ heap فإن هذه العملية يمكنها إصلاح الخلل ورفع العنصر الأكبر حتى مكانه المناسب أي: لو فرضاً افترضنا أن الخلل في الجذر فقط وباقي الأشجار الجزئية أدناه تحقق خاصية الـ heap، فتنفيذ هذه العملية لمرة واحدة على الجذر كفيل بإصلاح الـ HEAP بالكامل وجعله يحقق خاصيته

- في حال كانت الشجرة الثنائية ليست heap بالأساس، ستتم هذه العملية على العقد من الأسفل إلى الأعلى عقدةً عقدةً (سنعتبر أن كل الشجرة مختلة ونصلح الخلل من الأسفل إلى الأعلى لكل العقد "من الأوراق وحتى الجذر")، لما ذلك؟

لأننا كما لاحظنا ضمن العملية نحن فقط نقوم بمقارنة ابني عقدة ما مع العقدة والتبديل في حال كان أحد الأبناء ذو قيمة أكبر ثم سنقوم بالنزول إلى شجرة الابن الذي بدلنا معه أي نستدعي التابع عليه (في حال بدلنا في كل استدعاء سنقوم بنفس العملية حتى الوصول للأوراق أو حتى نصل لنقطة لا تغييرات فيها لكن هذا ليس كافياً كونه يمكن أن يكون هنالك قيم أكبر بالشجرة الجزئية التابعة لأحد الأبناء فلا بد لنا أن نضمن أن العنصر الموجود في الابن هو الأكبر في شجرته الجزئية وبالتالي نبدأ من الأسفل إلى الأعلى حتى نقوم برفع القيم الكبرى إلى الأعلى حتى الوصول إلى الجذر.



مثال مبسط على المشكلة

لو بدأنا بتطبيق العملية من الأعلى للأسفل

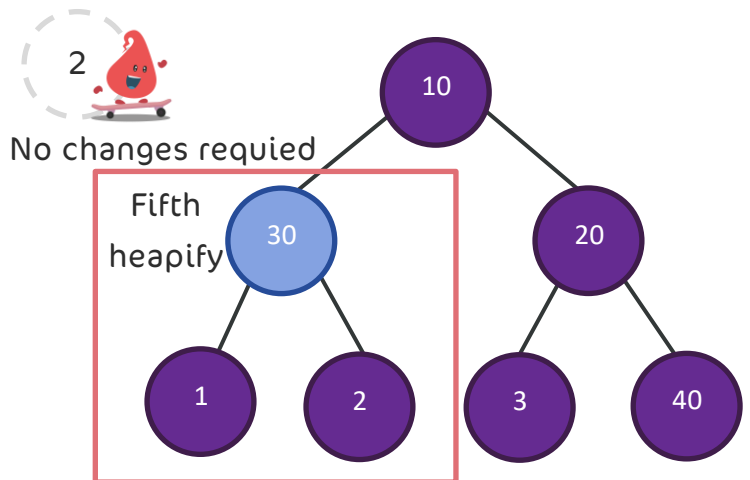
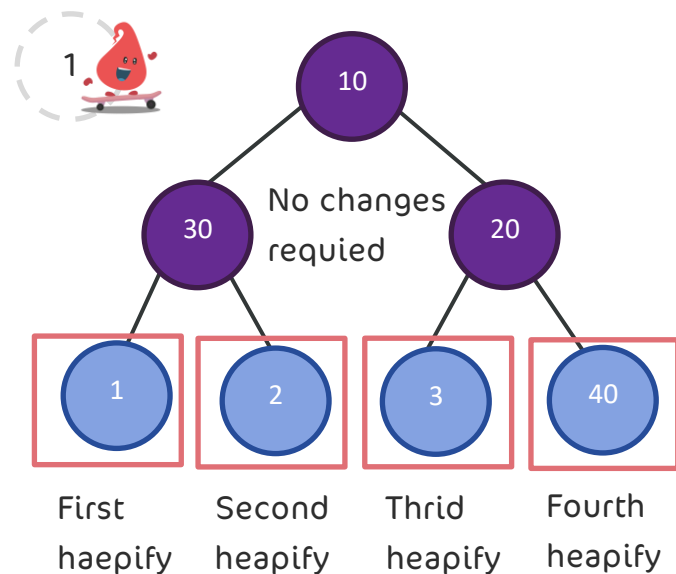
الآن مهما قمنا باستدعاء التابع على الأبناء وصولاً للأوراق

لن تصعد الـ 40 إلى الجذر وكناتج نهائي

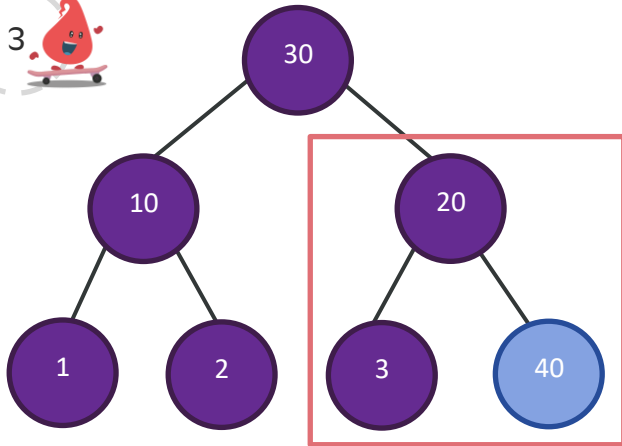
بعد استدعاء التابع على كل العقد الأبناء

سيكون الناتج هو الشجرة التالية وهذا ليس heap.

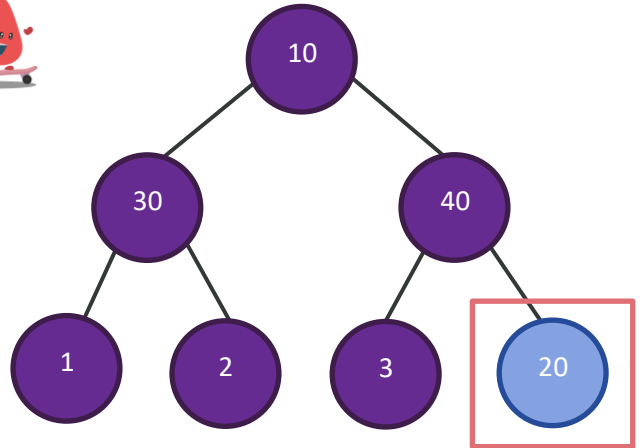
بينما سنعود لنفس الشجرة و ننفذ العمليات من الأسفل للأعلى



3

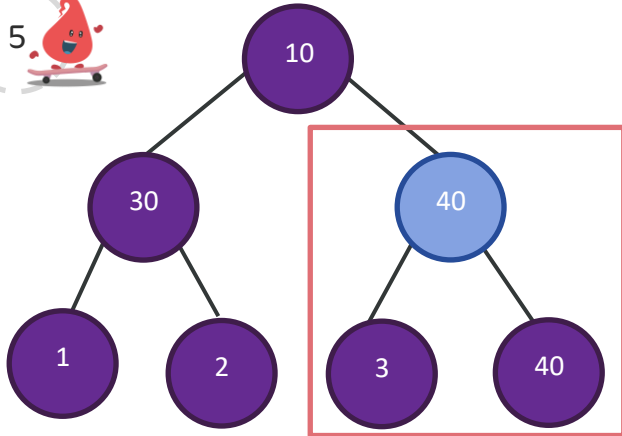


4

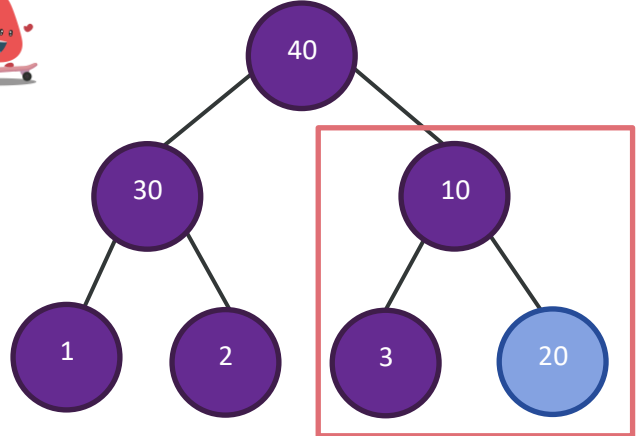


As we mentioned we call the function on the affected branch of the tree recursively (In this specific case no updates required so we stop here)

5

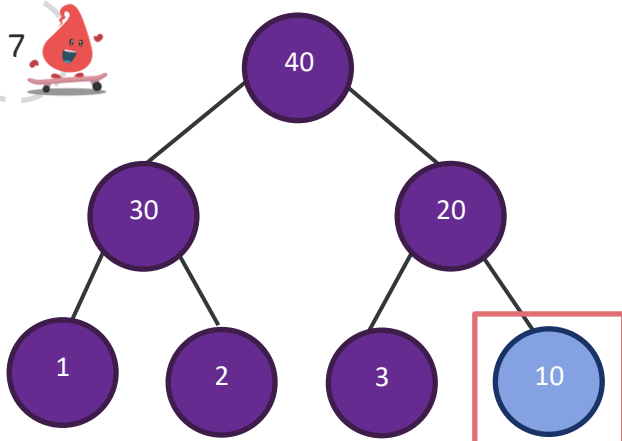


6



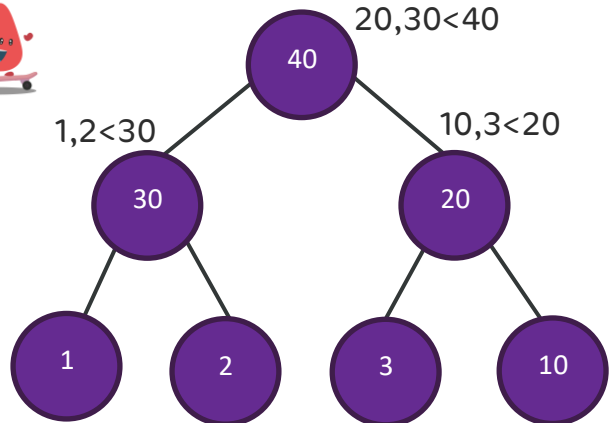
As we mentioned we call the function on the affected branch of the tree recursively (we need to swap 10 with 20 then call the function on the right child again)

7



No changes needed so we stop here

8



Leaf nodes are always valid heaps

Final result and it's a valid heap

ملاحظة هامة:

سنستدعي التابع كما قلنا من الأسفل إلى الأعلى ولكن يمكننا استثناء العقد الورقية كون العقد الورقية هي دوماً valid heaps ولن ننفذ أي عمليات عليها بحد ذاتها، لذلك يمكننا البدء باستدعاء التابع على أول عقدة غير ورقية (داخلية) ضمن الشجرة وصولاً إلى الجذر، يمكننا تحديد فهرس (index) أول عقدة تمتلك أبناء — أي أول عقدة غير ورقية — باستخدام العلاقة $(n / 2) - 1$ ، حيث n هو عدد العقد الكلي في الشجرة.

بعد فهم مبدأ العملية سنكتب code يقوم بما ذكرناه مسبقاً

```
import java.util.ArrayList;

public class Main {
    public static void heapify(ArrayList<Integer> arr,int i,int size) {
        // indexes setup
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        // if the left child is larger
        if (left<size&&arr.get(left)>arr.get(largest)){
            largest = left;
        }
        // if the right child is larger
        if (right < size && arr.get(right) > arr.get(largest)) {
            largest = right;
        }
        // if the largest element is one of the children of the root
        if (largest != i) {
            //swap it with the root
            int temp = arr.get(i);
            arr.set(i, arr.get(largest));
            arr.set(largest, temp);
            //call the heapify on the affected branch
            heapify(arr, largest, size);
        }
    }
}
```

1



```

public static void buildMaxHeap(ArrayList<Integer> arr) {
    int size = arr.size();
    //starting from the first non leaf node until the root of the tree
    for (int i = size / 2 - 1; i >= 0; i--) {
        //heapify each node
        heapify(arr, i, size);
    }
}

public static void main(String[] args) {
    ArrayList<Integer> arr = new ArrayList<>();
    arr.add(3);
    arr.add(1);
    arr.add(6);
    arr.add(5);
    arr.add(2);
    arr.add(4);
    System.out.println("before building max-heap " + arr);
    buildMaxHeap(arr);
    System.out.println("after building max-heap " + arr);
}
}

```

الآن بعد أن تحدثنا عن كيف نبني heap من مصفوفة معطاة فيها completed

binary tree ما تعقيد العملية السابقة بتصوركم؟

من النظرة الأولى يمكننا القيام بالتفكير، إن الـ heapify تقوم بعملية على عقدة ما نزولاً حتى الوصول إلى الأوراق وهذا بأسوأ حالة $O(h)$ حيث h هو ارتفاع العقدة التي نحن عندها، وكوننا نستدعي التابع على $n/2$ عقدة تقريباً (نهمل $1/2$ ونعتبرهم n عقدة) هذا يعني أننا نقوم بـ $n \cdot h$ عملية تقريباً ويمكن افتراض أن h هي $\log(n)$

لكن هذا ليس دقيقاً تماماً يمكن القيام بتحليل أدق للموضوع، كلما ابتعدنا عن الجذر كلما قل عدد العمليات المطلوبة (قيمة h تنقص) وعدد العقد الأبعد عن الجذر أكبر بكثير من التي هي أقرب له (يزداد عدد العقد بشكل أسّي كلما ابتعدنا عن الجذر)، سنجد بالتالي أن معظم العقد تحتاج عمليات معدودة حتى يتم إصلاحها بالتالي $O(1)$ و عدد قليل من العقد يحتاج تقريباً $\log(n)$ عملية حتى يتم إصلاحها، بالتالي التعقيد تقريباً $O(1)$ لعملية الـ heapify لو نظرنا إلى

العقد بشكل مجموعي لكل ارتفاع من الارتفاعات ، بالتالي نجد أن تعقيد بناء ال heap يصبح خطياً $O(n)$.
هذا الكلام مبرهن رياضياً سندرج البرهان كما هو من المرجع:

للإطلاع:

We can compute a simple upper bound on the running time of **BUILD-MAX-HEAP** as follows. Each call to **MAX-HEAPIFY** costs $O(\log n)$ time, and **BUILD-MAX-HEAP** makes $O(n)$ such calls. Thus, the running time is $O(n \log n)$. This upper bound, though correct, is not asymptotically tight.

We can derive a tighter bound by observing that the time for **MAX-HEAPIFY** to run at a node varies with the height of the node in the tree, and the heights of most nodes are small. Our tighter analysis relies on the properties that an n -element heap has height $\lceil \log n \rceil$ (see Exercise 6.1-2) and at most $\lceil n/2^{h+1} \rceil$ nodes of any height h (see Exercise 6.3-3).

The time required by **MAX-HEAPIFY** when called on a node of height h is $O(h)$, and so we can express the total cost of **BUILD-MAX-HEAP** as being bounded from above by

$$\sum_{h=0}^{\lceil \log n \rceil} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O(n) \sum_{h=0}^{\lceil \log n \rceil} \frac{h}{2^h}$$

We evaluate the last summation by substituting $x=1/2$ in the formula (A.8), yielding

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1 - 1/2)^2} = 2$$

Thus, we can bound the running time of **BUILD-MAX-HEAP** as

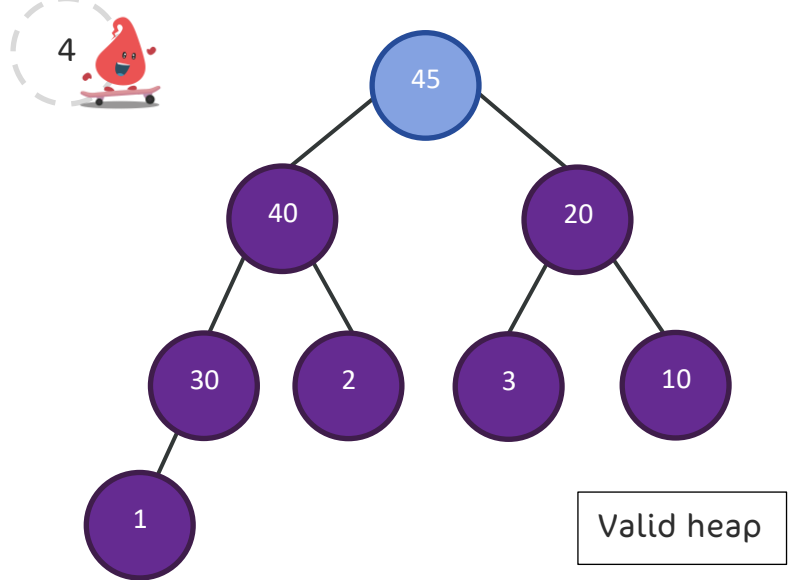
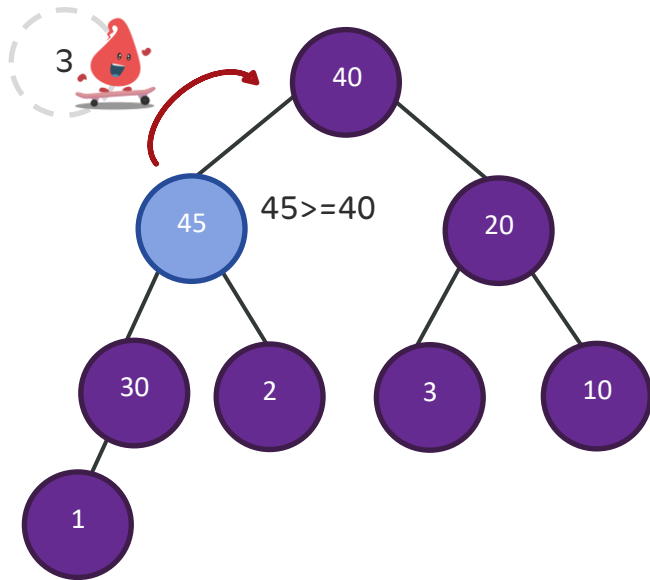
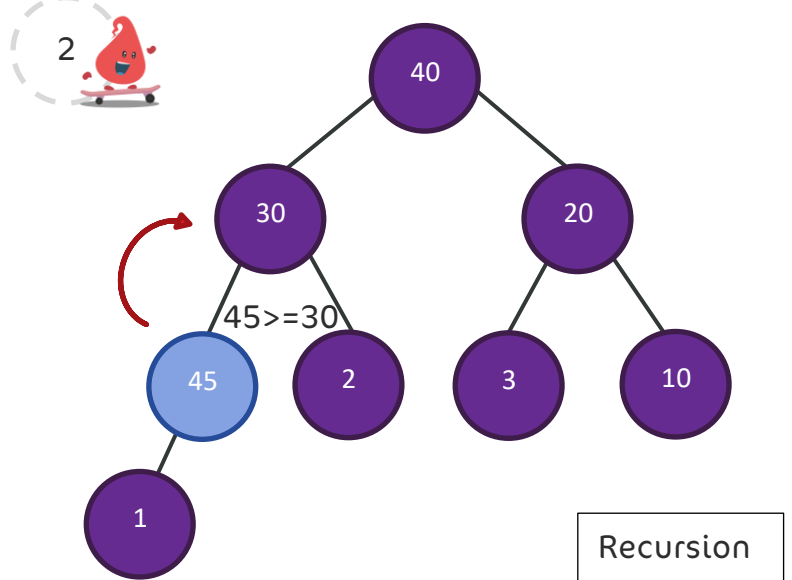
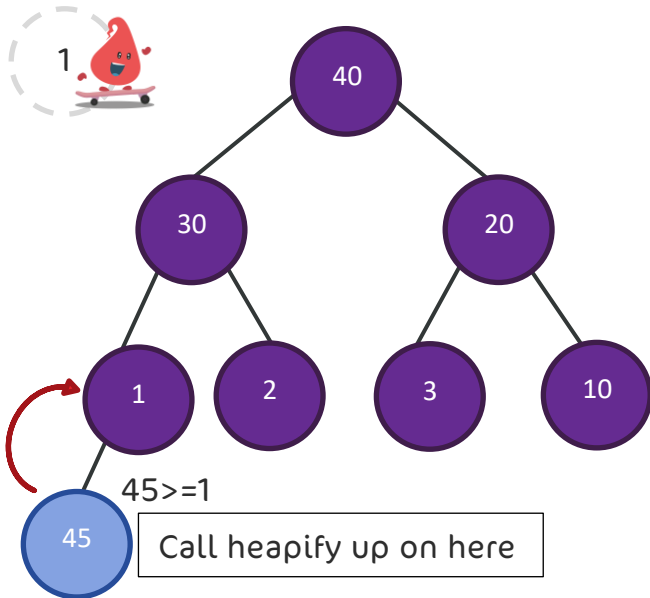
$$O(n) \sum_{h=0}^{\lceil \log n \rceil} \frac{h}{2^h} = O(n) \sum_{h=0}^{\infty} \frac{h}{2^h} = O(n)$$

العمليات التي يمكن تنفيذها على ال Heap

1. ال insert:

كيف نقوم بإدراج عقدة دون تخريب خاصية ال heap؟

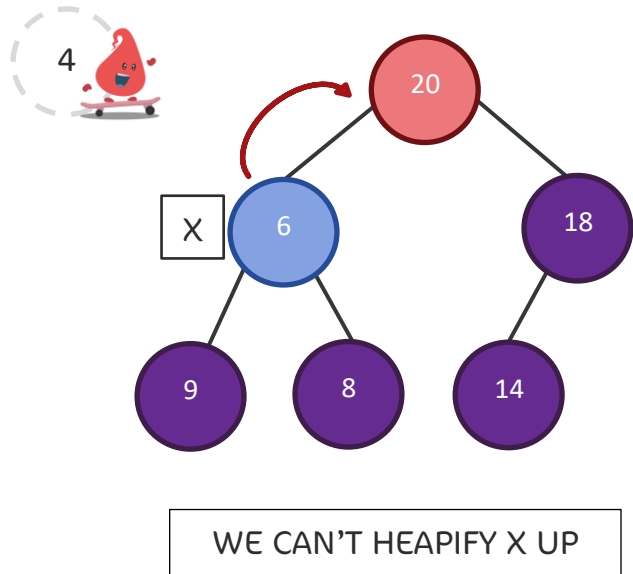
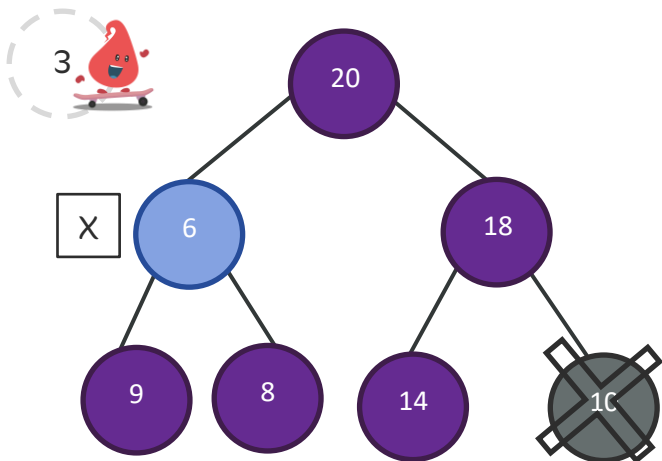
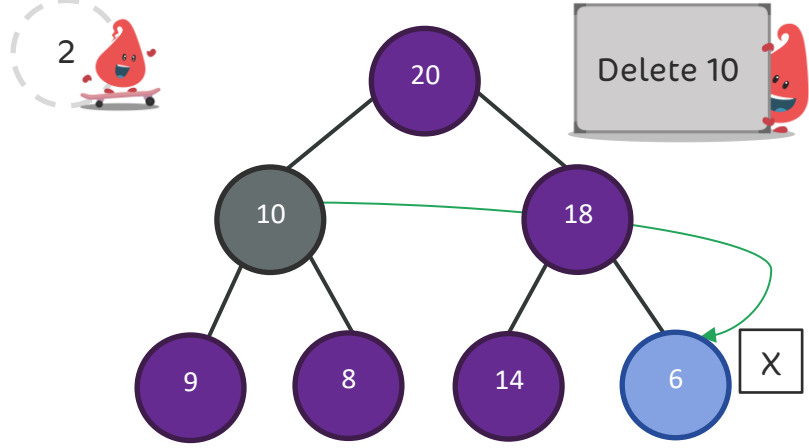
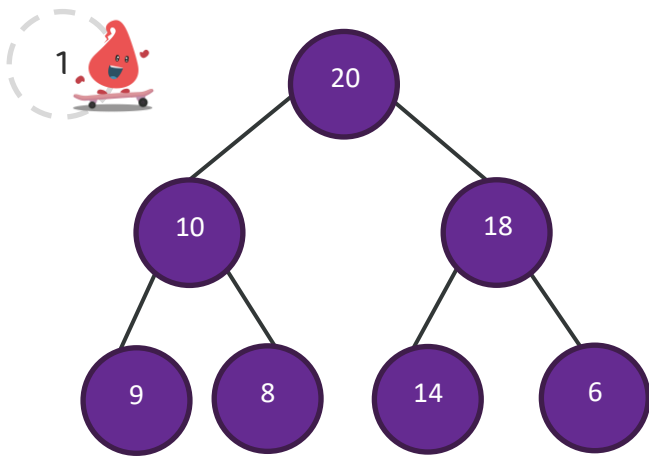
- بدايةً نقوم بإضافة العقدة الجديدة في نهاية المصفوفة التي لدينا.
- سنقوم بتعريف عملية جديدة مشابهة للعملية السابقة heapify سنسميها heapifyUp، كون العناصر التي في الشجرة بشكل عام تحقق خاصية ال heap إلا آخر عنصر (العنصر المدخل حديثاً)، لذلك هذه العملية مهمتها أن تقوم برفع العنصر المضاف حديثاً. (لنفترض أن لدينا max heap)
- تقارن الدالة هذا العنصر مع الأب الخاص به إن كان أكبر نبدل بينهم ونقوم بنفس العملية مع الأب وهكذا حتى نصل إلى الجذر، ويمكن أن نتوقف عندما لا يكون من المسموح عمل تبديل.

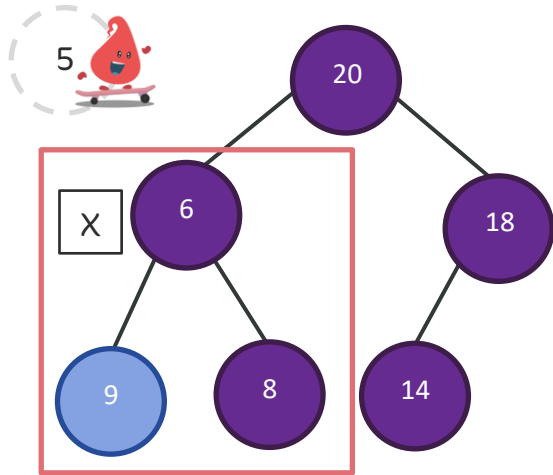


عند الحذف سنقوم بالتالي:

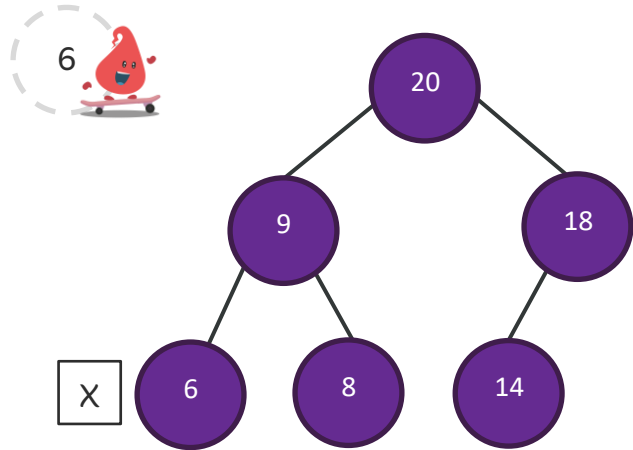
- نبدل العنصر الذي نريد حذفه بآخر عنصر بالمصفوفة وليكن X
- ثم نقوم بحذف العنصر الأخير (الحذف من الأخير أسهل)، لكن الآن العنصر X احتل مكان العنصر الذي حذفناه ولا يمكننا معرفة ما إذا كان مكانه صحيحاً أي: لدينا خلل في نقطة واحدة من الشجرة لحد الآن، فهذا العنصر إما يجب أن نرفعه للأعلى أو ننزله للأسفل. بالتالي نقوم بالتأكد هل يمكننا رفعه للأعلى باستخدام ال heapify up كونه الشجرة بشكل عام تحقق خاصية ال heap أعلى هذه النقطة التي صار فيها خلل:
 - إن لم يكن بإمكاننا عمل heapify up نقوم بعمل heapify طبيعية على العقدة X حتى نقوم بإصلاح الخلل بها وبشجرتها الجزئية إن وجد
 - إن لم يوجد خلل فتبقى العقدة مكانها، وبالتالي يعود ال HEAP لوضعه الطبيعي.

الآن لنبدأ بتمثيل ذلك:

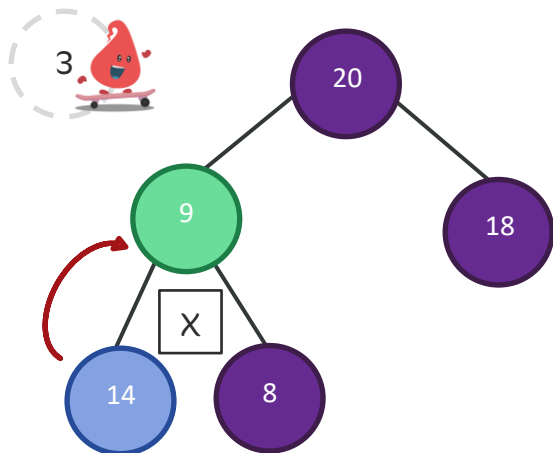
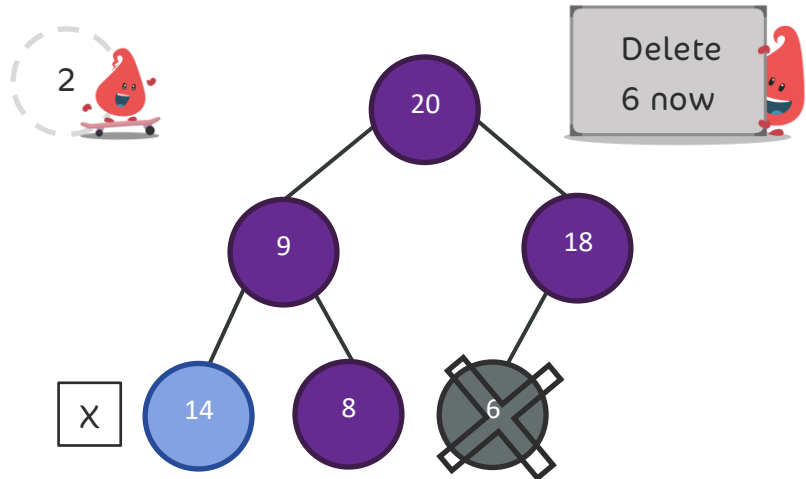
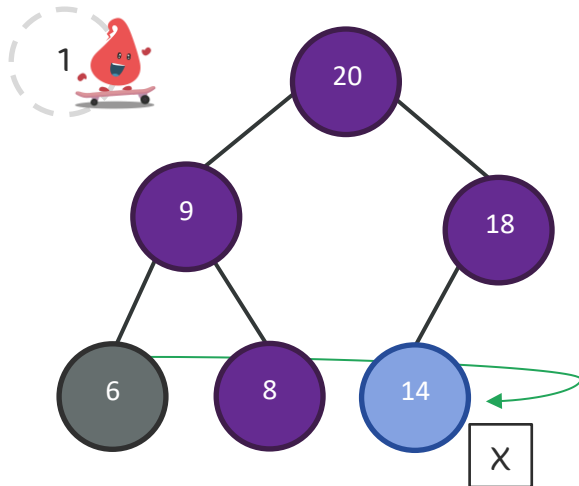




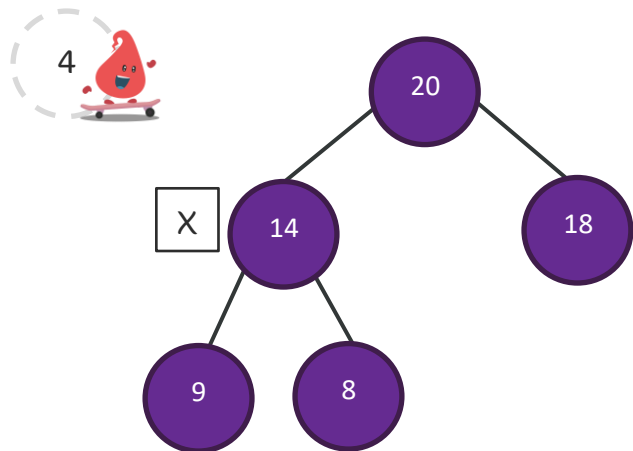
WE HEAPIFY X normally



Now we are done and as we see it's a valid heap



WE CAN HEAPIFY X UP



Now everything is alright no need to use normal heapify

الآن سنستعرض الكود و كيف يمكن لنا القيام بالعمليتين السابقتين الحذف و

الإضافة:

1

```
import java.util.ArrayList;
//class representing a max heap
class MaxHeap {
    // we will use an arraylist
    private ArrayList<Integer> heap=new
    ArrayList<>();
    //getting indexes of parent/left,right
    children (just to make it clear for you)
    private int parent(int i) {return (i - 1) / 2; }
    private int left(int i) {return 2 * i + 1; }
    private int right(int i) {return 2 * i + 2; }
    private void heapifyUp(int index) {
        // if we reached the root
        if (index == 0) return;
        //if the element is larger than it's parent then
        swap them and repeat the process on the
        parent
        if(heap.get(index)>heap.get(parent(index))){
            int temp = heap.get(index);
            heap.set (index,heap.get (parent (index)));
            heap.set(parent(index), temp);
            heapifyUp(parent(index));
        }
    }
    void heapify(int index) {
        // indexes and size setup
        int size = heap.size();
        int largest = index;
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        // if the left child is larger
        if(left<size&&heap.get(left)>heap.get(largest)){
            largest = left;
        }
```

2

```
// if the right child is larger
if (right < size &&heap.get(right) >
heap.get(largest)) {
    largest = right;
}
// if the largest element is one of the
children of the root
if (largest !=index) {
    //swap it with the root
    int temp = heap.get(index);
    heap.set(index, heap.get(largest));
    heap.set(largest, temp);
    //call the heapify on the affected
    branch
    heapify(largest);
}
}
public void insert(int value) {
    //we just add the value at the last index
    heap.add(value);
    // then we heapify it up
    heapifyUp(heap.size() - 1);
}
```



3

```

public void delete(int value){
    //we search for the desired value first
    int index = heap.indexOf(value);
    //if we don't find it just return
    if (index == -1) return;
    //we swap between the last element in the heap and the one we want to delete
    int lastIndex = heap.size()-1;
    int temp = heap.get(index);
    heap.set(index, heap.get(lastIndex));
    heap.set(lastIndex, temp);
    //we remove the last element from the array
    heap.remove(lastIndex);
    //this condition guarantees that the value which we deleted is not the last element itself
    {so we don't get out of bounds exception}
    //(no need to repair the heap anyway in this case)

```

5

```

if (index < heap.size()) {
    //if the element is larger than its
    parent we heapify up
    if(index>0&&heap.get(index) >
    heap.get(parent(index))){
        heapifyUp(index);
    }
    // else we heapify normally
    else {
        heapify(index);
    }
}
}
public void printHeap() {
    System.out.println(heap);
}

public static void main(String[] args) {
    MaxHeap h = new MaxHeap();
    h.insert(20);
    h.insert(10);
    h.insert(18);
    h.insert(9);
    h.insert(8);
    h.insert(14);
    h.insert(6);
    System.out.print("normal heap:");
    h.printHeap();
    h.delete(6);
    System.out.print("after deleting 6:");
    h.printHeap();
    h.delete(10);
    System.out.print("after deleting 10:");
    h.printHeap();
}
}

```

هذا هو ال heap بصورة عامة، وتعرفنا على العمليات الأساسية به (يمكن تطبيق ذلك على ال min heap بمجرد التلاعب بالشروط).

أهم استخدامات ال Heap

1. أشهر بنى المعطيات priority queue مبنية عليه

- تقوم على مبدأ أنك يمكنك استخراج أكبر عنصر / أصغر عنصر (حسب نوع ال heap) بشكل دائم ومن المتاح لنا الإضافة وحذف العنصر الأكبر / الأصغر
- يمكننا تعريف ال priority queue بناء على ال heap من خلال جعل الحذف يكون فقط على الجذر بحيث يأخذ المستخدم القيمة ونحذفها من المصفوفة الخاصة بال heap ونقوم بعملية إصلاح heapify على الجذر.

2. أحد خوارزميات الترتيب والتي تدعى بـ selection sort

- لنفترض أن لدينا مصفوفة تحتوي على n عنصر ونريد ترتيبها تصاعدياً
- نبحث عن العنصر الأصغر بينهم ونضعه في بداية المصفوفة
 - ثم نستثني أول عنصر ونبدأ من العنصر الثاني نبحث عن العنصر الأصغر في المصفوفة
 - ثم عند إيجاده نضعه كعنصر ثانٍ في المصفوفة
 - ثم نستثني أول عنصرين كذلك ونبحث عن الأصغر في باقي المصفوفة ونضعه كعنصر ثالث
 - بالتالي عند الانتهاء من تكرار العملية سنجد أن المصفوفة مرتبة
- هذا في أسوأ الأحوال يتطلب $O(n^2)$ حيث أننا لكل مكان في المصفوفة المكونة من n عنصر سنحتاج تقريباً n عملية حتى نبحث عن العنصر الأصغر (بحث خطي)، بالتالي هذا تعقيد سيء مقارنة بخوارزميات الترتيب الأكفأ



ما علاقة خوارزمية Selection sort بموضوعنا اليوم؟

- نلاحظ أننا نعتمد بصورة عامة على إيجاد العنصر الأصغر بشكل دوري و نستثني العناصر التي استخدمناها مسبقاً في هذه الخوارزمية ، و نلاحظ أننا من خلال ال min heap يمكننا تقديم أصغر عنصر ، و عند الانتهاء منه يمكننا حذفه و سيعيد ال min heap إصلاح نفسه و تقديم العنصر الأصغر الذي يليه و هذا بتعقيد $O(\log(n))$ (تعقيد ال heapify بالنسبة للجذر) ، و بالتالي يمكننا تحسين خوارزمية الترتيب السابقة من خلال استخدام ال heap ، فيصبح من أجل كل مكان في المصفوفة المكونة من n عنصر نحتاج $O(\log(n))$ عملية حتى نصل للعنصر الأصغر ، و بالتالي يصبح التعقيد الكلي $O(n \log(n))$
- نوع الترتيب الجديد هذا يسمى heap sort



THE END